Position & Flexbox

Taking Control of Layout — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser**, **VS Code**, and **DevTools** open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses matter more than being right.

Part 1 — Position: Taking Control of Where Things Go

So far, every element has just sat where the HTML puts it, one after another. What if you wanted to move something without changing the HTML order — like a badge in the corner of a card, or a header that stays on screen while you scroll?

TRY THIS · VS CODE

Create a new file `positions.html` and add boilerplate code to it. Have two `<div>` boxes `block-1` and `block-2`. Add padding, border and different background color. Have the background-color to the `body` as well. Keep all three background colors different.

```
body { ... }
.block-1 { ... }
.block-2 { ... }
```

```
<body>
    <div class='block-1'>A</div>
    <div class='block-2'>B</div>
</body>
```

Q1. Where do the boxes sit relative to each other? This default behavior actually has a name — any guess what it's called?

TRY THIS · VS CODE

Add `position: relative; top: 20px; left: 20px;` to `block-1`.

Q2. What happened to the box? Did the second box shift up to fill the gap it left behind, or is that gap still there? What do you think “relative” means here — relative to *what*?

TRY THIS · VS CODE

Now, just change `position: relative;` to `position: absolute;` for `block-1`.

Q3. Where did the box end up now? Did the second box move to fill the space this time, unlike with `relative`?

TRY THIS · VS CODE

Give `48px` padding to `body`. Make `block-1` have `top: 0; left: 0;`.

Q4. Can you analyze where the absolute element starts?

TRY THIS · VS CODE

Now, wrap both blocks in a parent `<div class='parent'>` and have `position: relative;` on the `parent` class.

```
body { ... }
.block-1 { ... }
.block-2 { ... }
.parent {
    position: relative;
}
```

```
<body>
    <div class='parent'>
        <div class='block-1'>A</div>
        <div class='block-2'>B</div>
    </div>
</body>
```

Q5. Where does the absolutely positioned box sit now — relative to the whole page, or relative to something closer?

TRY THIS · VS CODE

Now remove `position: relative;` from the `parent` class.

Q6. Where did the box jump back to? What does this confirm about which ancestor actually “counts” for an absolutely positioned element?

TRY THIS · VS CODE

Give `block-1` and `block-2` the same CSS `position: absolute; top: 50px; left: 50px;` so they overlap completely.

Q7. Which box appears on top by default? Now add `z-index: 1;` to `block-2` — what changed?

Q8. In your own words, describe `static`, `relative`, `absolute`, and `fixed` — and when you'd reach for each.

Part 2 — Flexbox: Arranging Things Without Fighting the Box Model

Remember centering something horizontally needing `margin: 0 auto;`, but only for block elements with a set width — and centering vertically being basically impossible without hacks? What if there were a display mode built specifically for arranging boxes in a row or column?

TRY THIS · VS CODE

Now create an HTML file with boilerplate, `flex-box.html`. A `<div class="container">` with three child `<div>` boxes inside, each with `48px` padding and different background colors.

```
body { ... }
.container { ... }
.block-1 { ... }
.block-2 { ... }
.block-3 { ... }
```

```
<body>
    <div class='container'>
        <div class='block-1'>A</div>
        <div class='block-2'>B</div>
        <div class='block-3'>C</div>
    </div>
</body>
```

Q9. How do the boxes currently sit — stacked, or side by side?

TRY THIS · VS CODE

Add `display: flex;` to the `container` class.

Q10. What changed immediately, without touching the children at all?

TRY THIS · VS CODE

Add `justify-content: center;`. Then try `space-between`, then `space-around`.

Q11. What happened to the three boxes along the row each time? How is `space-between` different from `space-around`?

TRY THIS · VS CODE

Create a new file, `fixed-nav.html`, with the usual boilerplate. Then add the content like this:

```
<body>
    <nav>My Web App</nav>
    <main>
        <article>
            <h2>ABCD</h2>
            <p>lorem350</p>
        </article>
        <article>
            <h2>XYZ</h2>
            <p>lorem350</p>
        </article>
        <article>
            <h2>MNOP</h2>
            <p>lorem350</p>
        </article>
        <article>
            <h2>FGGHIJKL</h2>
            <p>lorem350</p>
        </article>
    </main>
    <footer>
        &copy; My Web App – 2026
    </footer>
</body>
```

TRY THIS · VS CODE

Add some styles to make the navbar look different, then open `fixed-nav.html` with live server.

```
* {
    padding: 0;
    margin: 0;
    box-sizing: border-box;
}
main {
    padding: 24px;
    background-color: azure;
}
nav {
    padding: 24px;
    background-color: lightblue;
}
article {
    padding: 24px;
    background: lightgrey;
    border-bottom: 2px solid brown;
    margin: 48px;
}
footer {
    background-color: lightblue;
    padding: 24px;
}
```

TRY THIS · VS CODE

Now add this to the `nav` CSS: `position: fixed; top: 0; left: 0;`.

Q12. What happens to the nav bar while you scroll? How is `fixed` different from `absolute`?

Q13. How can you make the navbar stretch the whole screen?

Q14. How can you have a `48px` visible gap between the navbar and the first article?

TRY THIS · VS CODE

Come back to `flex-box.html`. Give the `container` class `height: 300px;` and then give different height and width to the blocks. Then add `align-items: center;` to the `container` class.

Q15. What happened to the boxes vertically? Which property controls the horizontal direction, and which controls vertical?

TRY THIS · VS CODE

Add `flex-direction: column;` to the same container, keeping `justify-content: center;` and `align-items: center;`.

Q16. Did `justify-content` and `align-items` swap which direction they control? What do you think `flex-direction` actually changes?

TRY THIS · VS CODE

Give the container `height: 100vh;` with `display: flex; justify-content: center; align-items: center;`. Comment out the other two boxes — just keep one box.

Q17. Where does the box end up? Did this just solve the vertical-centering problem you flagged as basically impossible at the start of this section?

Q18. In your own words: what problem does `display: flex` solve that you were struggling with before? What's the difference between `justify-content` and `align-items`?

Part 3 — Practice: Flexbox Froggy

TRY THIS · BROWSER

Open **flexboxfroggy.com** and start from Level 1. Read the description given in each level. If you know the property, try to write the code. If not, then read about it. Keep writing the property names you come to know in the box below. We'll discuss each new property you get stuck on.

Q19. Which are the intuitive properties you figured out in this activity?

Q20. Which are the non-intuitive properties you figured out in this activity?

Q21. Which are the confusing properties you think you need to practice more?

Q22. After this activity, are you able to appreciate the use of flexbox?